

数据结构与算法 K06 课堂练习

单词最小编辑距离的递归解法

张植竣 1900011604 北京大学物理学院天文学系

2020 年 3 月 31 日

源代码

```
1  def dpWordEdit(original, target, oplist):
2      score = 0
3      operations = []
4
5      def Insert(s):
6          return oplist['insert'] # 添加一个字母
7
8      def Delete(t):
9          return oplist['delete'] # 删除一个字母
10
11     def Replace(m, n):
12         if m == n:
13             return oplist['copy'] # 替换一个相同的字母 == 复制一个字母
14         else:
15             return oplist['insert'] + oplist['delete'] # 替换一个不同的字母 ↔
16                 == 添加一个字母，再删除一个字母
17
18     originallist = list(original) # 源单词列表
19     targetlist = list(target) # 目标单词列表
20
21     MinScore = {}
22     # 字典，储存递归过程中的中间步骤
23     solution = {(i, j):[] for i in range(len(originallist) + 1)
24                 for j in range(len(targetlist) + 1)}
25     # 二维字典，储存由源单词中前i个字母变为目标单词中前j个字母所需要的最少操作↔
26     # 过程
27
28     def minscore(i, j): # 双重递归，在MinScore中填写一个二维递归字典
29
30         if i == 0 and j != 0:
31             sco = j * oplist['insert']
```

```

30         MinScore[(i, j)] = sco
31         solution[(i, j)] = solution[(i, j - 1)][:]
32         solution[(i, j)].append('insert {0}'.format(targetList[j - 1]))
33         return sco
34     # 递归字典的首行
35
36     elif j == 0 and i != 0:
37         sco = i * oplist['delete']
38         MinScore[(i, j)] = sco
39         solution[(i, j)] = solution[(i - 1, j)][:]
40         solution[(i, j)].append('delete {0}'.format(originalList[i - 1]))
41         return sco
42     # 递归字典的首列
43
44     elif i == 0 and j == 0:
45         sco = 0
46         MinScore[(i, j)] = sco
47         solution[(i, j)] = []
48         return sco
49     # 递归字典的第一个元素
50
51     elif (i, j) in MinScore: # (i, j)元的值已经被储存在MinScore中，则直接↔
        返回
52         sco = MinScore[(i, j)]
53         return sco
54
55     else:
56         sco = min(minscore(i, j - 1) + Insert(targetList[j - 1]),
57                  # 源单词比目标单词少一个字母，添加该字母，记录分值
58                  minscore(i - 1, j - 1) + Replace(originalList[i - 1], ↔
                    targetList[j - 1]),
59                  # 源单词与目标单词同时增加一个字母，比较这两个字母是否相↔
                    同，
60                  # 选择是复制该字母或是添加目标字母，再删除原字母，记录相↔
                    应分值
61                  minscore(i - 1, j) + Delete(originalList[i - 1]))
62         # 源单词比目标单词多一个字母，删去该字母，记录分值
63
64         if sco == minscore(i, j - 1) + Insert(targetList[j - 1]):
65             solution[(i, j)] = solution[(i, j - 1)][:]
66             solution[(i, j)].append('insert {0}'.format(targetList[j - 1]))↔
                )
67         # 源单词比目标单词少一个字母，添加该字母，记录操作
68
69         elif sco == minscore(i - 1, j - 1) + Replace(originalList[i - 1], ↔
            targetList[j - 1]):
70             solution[(i, j)] = solution[(i - 1, j - 1)][:]
71             if originalList[i - 1] == targetList[j - 1]:

```

```

72         solution[(i, j)].append('copy {0}'.format(targetList[j - ←
73             1]))
74         # 源单词与目标单词同时增加一个字母，两个字母相同，复制该字←
75         母，记录操作
76     else:
77         solution[(i, j)].append('insert {0}'.format(targetList[j -←
78             1]))
79         solution[(i, j)].append('delete {0}'.format(originalList[i←
80             - 1]))
81         # 源单词与目标单词同时增加一个字母，两个字母不同，添加目标←
82         字母，再删除原字母，记录操作
83
84     else:
85         solution[(i, j)] = solution[(i - 1, j)][:]
86         solution[(i, j)].append('delete {0}'.format(originalList[i - ←
87             1]))
88         # 源单词比目标单词多一个字母，删去该字母，记录操作
89
90     MinScore[(i, j)] = sco
91     return sco
92
93     score = minscore(len(originalList), len(targetList)) # 最终分值
94     operations = solution[(len(originalList), len(targetList))] # 最终操作,即←
95     solution表中的最后一个元素
96     return score, operations
97
98 # 检验
99 print("===== 单词最小编辑距离问题 =====")
100 oplist = [{'copy': 5, 'delete': 20, 'insert': 20},
101            {'copy': 5, 'delete': 10, 'insert': 15},
102            {'copy': 10, 'delete': 25, 'insert': 20}]
103 originalWords = ["cane", "sheep", "algorithm", "debug", "difficult", "←
104     directory", "wonderful"]
105 targetWords = ["new", "sleep", "alligator", "release", "sniffing", "framework"←
106     , "terrific"]
107
108 import time
109 a = time.time()
110 for i in range(len(oplist)): # 可以同时三种分值设置进行检验
111     for j in range(len(originalWords)):
112         score, operations = dpWordEdit(originalWords[j], targetWords[j], ←
113             oplist[i])
114         print(score)
115         print(operations)
116     print('')
117 b = time.time()
118 print('Total time used:%.4fs'%(b-a)) # 计时程序

```

输出结果截图

```
Python 3.8.1 Shell
File Edit Shell Debug Options Window Help
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 22:39:24) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\17732\Desktop\K06.py =====
===== 单词最小编辑距离问题 =====
70
['delete c', 'delete a', 'copy n', 'copy e', 'insert w']
60
['copy s', 'delete h', 'insert l', 'copy e', 'copy e', 'copy p']
185
['copy a', 'copy l', 'insert l', 'insert i', 'copy g', 'insert a', 'insert t', 'copy o', 'copy r', 'delete i', 'delete t', 'delete h', 'delete m']
205
['delete d', 'insert r', 'copy e', 'delete b', 'delete u', 'delete g', 'insert l', 'insert e', 'insert a', 'insert s', 'insert e']
200
['delete d', 'insert s', 'insert n', 'copy i', 'copy f', 'copy f', 'copy i', 'delete c', 'delete u', 'delete l', 'delete t', 'insert n', 'insert g']
220
['delete d', 'delete i', 'insert f', 'copy r', 'insert a', 'insert m', 'copy e', 'delete c', 'delete t', 'insert w', 'copy o', 'copy r', 'delete y', 'insert k']
235
['delete w', 'delete o', 'delete n', 'delete d', 'insert t', 'copy e', 'copy r', 'insert r', 'insert i', 'copy f', 'delete u', 'delete l', 'insert i', 'insert c']

45
['delete c', 'delete a', 'copy n', 'copy e', 'insert w']
45
['copy s', 'delete h', 'insert l', 'copy e', 'copy e', 'copy p']
125
['copy a', 'copy l', 'insert l', 'insert i', 'copy g', 'insert a', 'insert t', 'copy o', 'copy r', 'delete i', 'delete t', 'delete h', 'delete m']
135
['delete d', 'insert r', 'copy e', 'delete b', 'delete u', 'delete g', 'insert l', 'insert e', 'insert a', 'insert s', 'insert e']
130
['delete d', 'insert s', 'insert n', 'copy i', 'copy f', 'copy f', 'copy i', 'delete c', 'delete u', 'delete l', 'delete t', 'insert n', 'insert g']
145
['delete d', 'delete i', 'insert f', 'copy r', 'insert a', 'insert m', 'copy e', 'delete c', 'delete t', 'insert w', 'copy o', 'copy r', 'delete y', 'insert k']
150
['delete w', 'delete o', 'delete n', 'delete d', 'insert t', 'copy e', 'copy r', 'insert r', 'insert i', 'copy f', 'delete u', 'delete l', 'insert i', 'insert c']

90
['delete c', 'delete a', 'copy n', 'copy e', 'insert w']
85
['copy s', 'delete h', 'insert l', 'copy e', 'copy e', 'copy p']
230
['copy a', 'copy l', 'insert l', 'insert i', 'copy g', 'insert a', 'insert t', 'copy o', 'copy r', 'delete i', 'delete t', 'delete h', 'delete m']
230
['delete d', 'insert r', 'copy e', 'delete b', 'delete u', 'delete g', 'insert l', 'insert e', 'insert a', 'insert s', 'insert e']
245
['delete d', 'insert s', 'insert n', 'copy i', 'copy f', 'copy f', 'copy i', 'delete c', 'delete u', 'delete l', 'delete t', 'insert n', 'insert g']
265
['delete d', 'delete i', 'insert f', 'copy r', 'insert a', 'insert m', 'copy e', 'delete c', 'delete t', 'insert w', 'copy o', 'copy r', 'delete y', 'insert k']
280
['delete w', 'delete o', 'delete n', 'delete d', 'insert t', 'copy e', 'copy r', 'insert r', 'insert i', 'copy f', 'delete u', 'delete l', 'insert i', 'insert c']

Total time used: 0.8543s
```